How-To Library

Informatica

Performance Tuning Guidelines for Databricks Connector

© Copyright Informatica LLC 2020, 2025. Informatica, the Informatica logo, and Informatica Cloud Data Integration are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Abstract

You can use Databricks Connector to connect to Databricks Lake. This article describes general reference guidelines to help you tune the performance of Databricks Connector in Cloud Data Integration. It also describes the best practices required when you run a mass ingestion task to transfer tables from a Databricks source to a Microsoft Azure Data Lake Store Gen2 target and transfer Parquet files from a Microsoft Azure Data Lake Store Gen2 source to a Databricks target.

Supported Versions

- Informatica Cloud® Data Integration Databricks Connector

Table of Contents

Overview. 2
Tune the hardware. 2
CPU frequency. 3
NIC card ring buffer size. 3
Tune the heap size for the Secure Agent. 5
Best practices for a mass ingestion task. 5
Scalability. 5

Overview

You can configure a mass ingestion task using Databricks to transfer tables from a Databricks source to a Microsoft Azure Data Lake Store Gen2 target and transfer Parquet files from a Microsoft Azure Data Lake Store Gen2 source to a Databricks target.

Performance tuning is an iterative process to analyze the performance, use guidelines to estimate and define parameters that impact the performance, and monitor and adjust the results as required.

When you run a mass ingestion task, you can optimize the performance by tuning the following parameters:

- Hardware
- Heap Size

Note: The performance testing results listed in this article are based on observations in an internal Informatica environment using data from real-world scenarios. The performance of Databricks Connector might vary based on individual environments and other parameters even when you use the same data.

Tune the hardware

You can tune the following hardware parameters to optimize the performance of the machine where the Secure Agent runs:

- CPU Frequency
- NIC Card ring buffer size

CPU frequency

Dynamic frequency scaling adjusts the frequency of the processor on-the-fly either for power savings or to reduce heat. Ensure that the CPU operates at least at the base frequency.

When CPUs are underclocked, where they run below the base frequency, the performance degrades by 30% to 40%. Informatica recommends that you work with your IT system administrator to ensure that all the nodes on the cluster are configured to run at their supported base frequency.

To tune the CPU frequency for Intel multicore processors, perform the following steps:

1. Run the lscpu command to determine the current CPU frequency, base CPU frequency, and the maximum CPU frequency that the processor supports.
2. Request your system administrator to perform the following tasks:
   - Increase the CPU frequency to the supported base frequency.
   - Change the power management setting to **OS Control** at the BIOS level.
3. Run CPU-intensive tests to monitor the CPU frequency in real time and adjust the frequency for improved performance. On RedHat Linux operating systems, you can install a monitoring tool such as cpupower.
4. Work with your IT department to ensure that the CPU frequency and power management settings are persisted even for future system reboots.

NIC card ring buffer size

NIC configuration is a key factor in network performance tuning. When you deal with large volumes of data, it is crucial that you tune the Receive (RX) and Transmit (TX) ring buffer size. The ring buffers contain descriptors or pointers to the socket kernel buffers that hold the packet data.

You can run the ethtool command to determine the current configuration.

For example, run the following command:

```
# ethtool -g eth0
```

The following section shows a sample output:

```
Ring parameters for eth0:

Pre-set maximums:

RX: 2040

RX Mini: 0

RX Jumbo: 8160

TX: 255


Current hardware settings:

RX: 255

RX Mini: 0

RX Jumbo: 0

TX: 255
```

The **Pre-set maximums** section shows the maximum values that you can set for each parameter. The **Current hardware settings** section shows the current configuration details.

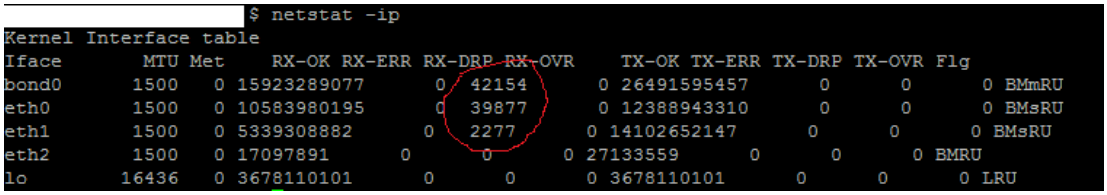
A long buffer size leads to low latency. However, low latency comes at the cost of throughput. For greater throughputs, you must configure large buffer ring sizes for RX and TX.

Informatica recommends that you use the ethtool command to determine the current hardware settings and the maximum supported values. Then, set the values based on the maximum values that are supported for each operating system. For example, if the maximum supported value for RX is 2040, you can use the following ethtool command to set the RX value to 2040:

```
# ethtool -G eth0 RX 2040
```

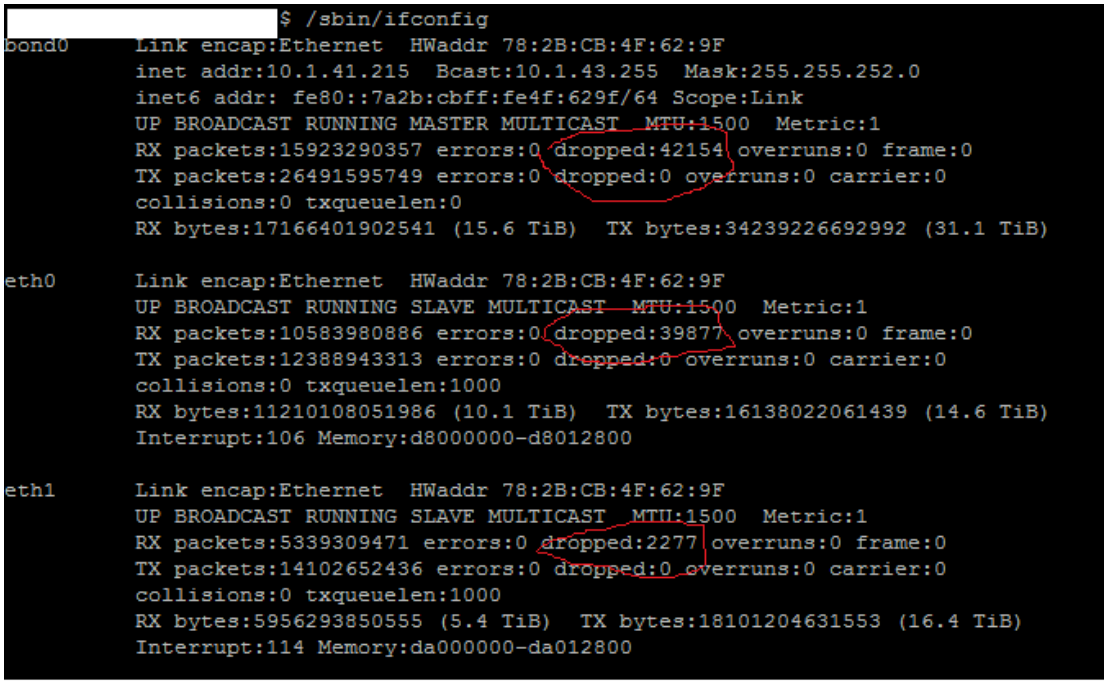
If you set a low ring buffer size for data transfer, packets might get dropped. To find out if packets were dropped, you can use the netstat and ifconfig commands.

The following image shows a sample output of the `netstat` command:



The RX-DRP column indicates the number of packets that were dropped. Set the RX value such that no packets get dropped and the RX-DRP column shows the values as 0. You might need to test several values to optimize the performance.

The following image shows a sample output of the `ifconfig` command:



The status messages indicate the number of packets that were dropped.

Tune the heap size for the Secure Agent

To increase the performance and avoid run-time environment memory issues, configure the Java heap size for the Informatica Intelligent Cloud Services Secure Agent.

1. Log in to Informatica Intelligent Cloud Services.
2. Select **Administrator** > **Runtime Environments**.
3. On the **Runtime Environments** page, select the Secure Agent for which you want to increase the memory from the list of available Secure Agents.
4. On the upper-right corner of the page, click **Edit**.
5. In the **System Configuration Details** section, select the **Type** as **DTM** for the Data Integration Server.
6. Enter **JVMOption1** as **-Xmx1024m**.
7. Click **Save**.

Best practices for a mass ingestion task

Use the following best practices when you configure a mass ingestion task to transfer Parquet files from a Microsoft Azure Data Lake Store Gen2 source to a Databricks target:

- When the Microsoft Azure Data Lake Store Gen2 source contains large file or files, Informatica recommends that you use a cluster with high configuration with more number of nodes with higher CPU and memory.
- When the Microsoft Azure Data Lake Store Gen2 source contains multiple files, Informatica recommends that you specify a batch size that is the same as the number of files in the Microsoft Azure Data Lake Store Gen2 source. This ensures that the Secure Agent avoids the time overhead of creating multiple spark tasks for each file in multiple batches.

Scalability

In scalability tests, the execution time is observed by varying the volume of data and keeping the following values constant:

- Mass ingestion task properties
- Disk space of the Secure Agent hosted virtual machine
- Size of the cluster

When the increase in the volume of the data is linear to the increase in the execution time, the performance is linearly scalable,

- When you transfer table(s) from a Databricks source, the execution time scales linearly with increasing volume of data in a single Databricks table and also by increasing number of tables in the Databricks source.
- When you transfer parquet file(s) to a Databricks target, the execution time scales linearly with increasing volume of data in a single Microsoft Azure Data Lake Store Gen2 parquet file and also by increasing number of parquet files in the Microsoft Azure Data Lake Store Gen2 source.

Author

Anupam Nayak

Acknowledgements

The author would like to acknowledge Manav Khurana for his technical assistance.